

03b_Spark_ETL_Basico

March 11, 2026

1 Práctica 0: Introducción a la Arquitectura Medallón con Spark

Dataset: Supermarket Sales (Ligero, ~1000 filas) **Tecnología:** Apache Spark (Modo Local) sobre HDFS

Este cuaderno ilustra el flujo de trabajo completo de Ingeniería de Datos utilizando la **Arquitectura Medallón** (Medallion Architecture). Es el estándar de la industria para organizar los lagos de datos (Data Lakes) en tres capas lógicas de madurez:

- **Capa Bronce (Raw):** Ingesta (Extract & Load). Contiene los datos crudos originales, tal cual vienen de la fuente.
- **Capa Plata (Silver):** Transformación Principal. Datos limpios, estandarizados, sin nulos y con tipos correctos, guardados en formato columnar (Parquet).
- **Capa Oro (Gold):** Lógica de Negocio. Transformación de negocio (agregados, KPIs, modelos para BI).

Objetivos: 1. Ingestar datos crudos a la Capa Bronce en formato CSV. 2. Limpiar e inferir el esquema para guardar los datos listos en la Capa Plata. 3. Calcular los ingresos totales por línea de producto y guardarlos en la Capa Oro.

```
[1]: # 1. Iniciar Sesión Spark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, desc

spark = SparkSession.builder \
    .appName("Practica_Supermercado_Spark_Medallion") \
    .master("local[*]") \
    .getOrCreate()

print("Spark inicializado y listo para trabajar en capas.")
```

Spark inicializado y listo para trabajar en capas.

1.0.1 Capa Bronce (Raw): Ingesta de Datos (E + L) y Transformación Mínima

Vamos a descargar un archivo CSV crudo con el histórico de ventas (usando Python básico para evitar bloqueos) y lo guardaremos intacto en nuestro lago de datos HDFS.

```
[2]: import urllib.request
import os
```

```
# Descargamos directamente desde un repositorio de Github Público para evitar
↳ errores 403 de Kaggle
# https://github.com/sushantag9/Supermarket-Sales-Data-Analysis/blob/master/
↳ supermarket_sales%20-%20Sheet1.csv
url = "https://raw.githubusercontent.com/sushantag9/
↳ Supermarket-Sales-Data-Analysis/master/supermarket_sales%20-%20Sheet1.csv"
os.makedirs("supermarket_sales", exist_ok=True)
archivo_local = "supermarket_sales/sales.csv"

urllib.request.urlretrieve(url, archivo_local)
print("¡Descarga finalizada con éxito!")
```

¡Descarga finalizada con éxito!

```
[3]: # Ingesta a la Capa Bronce distribuida en HDFS
!hdfs dfs -mkdir -p /data/bronze/supermarket/
!hdfs dfs -put -f supermarket_sales/*.csv /data/bronze/supermarket/
```

1.0.2 Capa Plata (Silver): Transformación Principal

Leemos el archivo crudo desde la capa Bronce, realizamos la estandarización (inferir esquema), limpiamos valores nulos, y lo guardamos comprimido de forma ultra-eficiente (Parquet) para las futuras analíticas.

```
[4]: # 1. Leemos de Bronce (CSV)
ruta_bronce = "hdfs://namenode:9000/data/bronze/supermarket/*.csv"
df_bronce = spark.read.option("header", "true").option("inferSchema", "true").
↳ csv(ruta_bronce)
print("Esquema bronze")
df_bronce.printSchema()

# 2. Transformación principal: limpieza, validación y estandarización de
↳ columnas
df_plata_casi_lista = df_bronce.na.drop()
# Mantenemos consistencia removiendo espacios en los nombres de columnas
df_plata = df_plata_casi_lista.withColumnRenamed("Product line",
↳ "product_line").withColumnRenamed("Total", "total_sales")

# 3. Guardamos en la Capa Plata formateado nativamente para Big Data (Parquet)
ruta_plata = "hdfs://namenode:9000/data/silver/supermarket/"
df_plata.write.mode("overwrite").parquet(ruta_plata)
print("Esquema planta")
df_plata.printSchema()
```

Esquema bronze
root

```

|-- Invoice ID: string (nullable = true)
|-- Branch: string (nullable = true)
|-- City: string (nullable = true)
|-- Customer type: string (nullable = true)
|-- Gender: string (nullable = true)
|-- Product line: string (nullable = true)
|-- Unit price: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Tax 5%: double (nullable = true)
|-- Total: double (nullable = true)
|-- Date: string (nullable = true)
|-- Time: timestamp (nullable = true)
|-- Payment: string (nullable = true)
|-- cogs: double (nullable = true)
|-- gross margin percentage: double (nullable = true)
|-- gross income: double (nullable = true)
|-- Rating: double (nullable = true)

```

Esquema planta

root

```

|-- Invoice ID: string (nullable = true)
|-- Branch: string (nullable = true)
|-- City: string (nullable = true)
|-- Customer type: string (nullable = true)
|-- Gender: string (nullable = true)
|-- product_line: string (nullable = true)
|-- Unit price: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Tax 5%: double (nullable = true)
|-- total_sales: double (nullable = true)
|-- Date: string (nullable = true)
|-- Time: timestamp (nullable = true)
|-- Payment: string (nullable = true)
|-- cogs: double (nullable = true)
|-- gross margin percentage: double (nullable = true)
|-- gross income: double (nullable = true)
|-- Rating: double (nullable = true)

```

1.0.3 Capa Oro (Gold): Transformación de Negocio

En esta capa leemos los datos ya estandarizados de la capa Plata y creamos agregaciones u operaciones complejas. Estos son los datos finales (Data Marts) que consumiría tu equipo de Business Intelligence (PowerBI/Tableau).

```

[5]: # 1. Leemos directamente de la Capa Plata (Nota: leer un Parquet es rapidísimo)
df_silver_ready = spark.read.parquet(ruta_plata)

```

```

# 2. Convertimos el detalle transaccional en un modelo agregado (KPI): ¿Qué
↳ línea de productos genera más ingresos totales?
df_oro = df_silver_ready.groupBy("product_line").agg(sum("total_sales").
↳ alias("ingresos_totales_kpi")).orderBy(desc("ingresos_totales_kpi"))

# 3. Guardamos nuestro KPI final en la Capa Oro listos para consumo
ruta_oro = "hdfs://namenode:9000/data/gold/kpi_ventas_producto/"
df_oro.write.mode("overwrite").parquet(ruta_oro)

df_oro.show()

```

```

+-----+-----+
|      product_line|ingresos_totales_kpi|
+-----+-----+
| Food and beverages| 56144.844000000005|
| Sports and travel| 55122.826499999996|
|Electronic access...| 54337.531500000005|
| Fashion accessories|          54305.895|
| Home and lifestyle| 53861.913000000001|
| Health and beauty| 49193.739000000016|
+-----+-----+

```

```
[6]: spark.stop()
```

```
[ ]:
```